

Computer Architecture Study Guide:

Memory System, Cache Memory, and Virtual Memory

คู่มืออ่านสอบระบบหน่วยความจำ แคช และหน่วยความจำเสมือน

เรียบเรียงใหม่จาก Lecture 9_MemorySystem, Lecture 10_CacheMemory, Lecture 11_VirtualMemory, Lecture 11_extended และสไลด์ที่เกี่ยวข้อง

TH/EN bilingual terminology / Tutor-style exam preparation

เป้าหมายของเอกสาร: อ่านจากพื้นฐานไปจนทำข้อสอบได้จริง เข้าใจภาพรวม Memory Hierarchy, Main Memory, Cache และ Virtual Memory พร้อมสูตรและโจทย์ฝึกครบชุด

สารบัญ (Table of Contents)

1. 1. บทนำ: ทำไมระบบหน่วยความจำจึงสำคัญ
2. 2. บทที่ 1 ภาพรวมระบบหน่วยความจำ (Memory System Overview)
3. 3. บทที่ 2 Main Memory และการเชื่อมต่อกับ CPU
4. 4. บทที่ 3 RAM และ ROM
5. 5. บทที่ 4 Cache Memory
6. 6. บทที่ 5 Cache Mapping
7. 7. บทที่ 6 Cache Performance
8. 8. บทที่ 7 Virtual Memory
9. 9. บทที่ 8 Paging และ Page Table
10. 10. บทที่ 9 เชื่อม Cache กับ Virtual Memory
11. 11. บทที่ 10 สรุปใหญ่ก่อนสอบ
12. 12. แนวข้อสอบพร้อมเฉลย
13. 13. Final Cheat Sheet

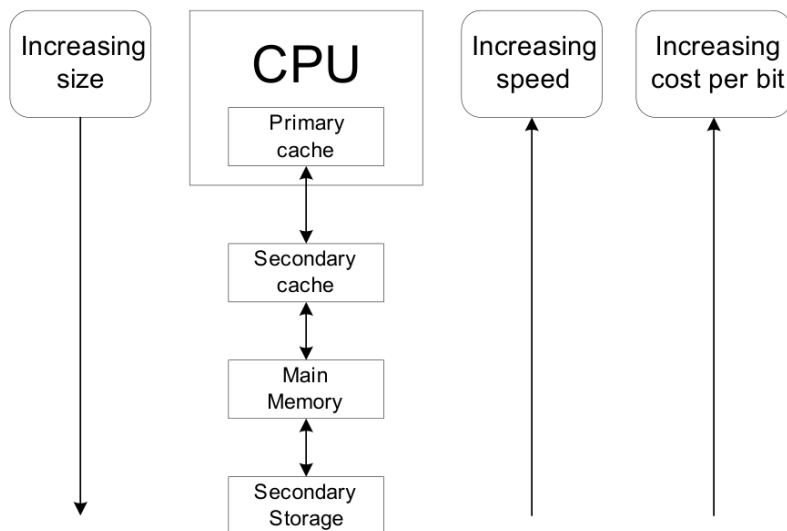
บทนำ: ทำไมระบบหน่วยความจำจึงสำคัญ

ในคอมพิวเตอร์สมัยใหม่ CPU สามารถประมวลผลได้เร็วมาก แต่ข้อมูลและคำสั่งต้องถูกนำมาจากหน่วยความจำก่อนเสมอ ดังนั้นถ้าหน่วยความจำตอบสนองช้า CPU จะต้องรอ แม้ตัว CPU จะเร็วแค่ไหนก็ตาม

English summary: A fast CPU can still be slowed down by memory latency because instructions and data must be fetched before execution.

แนวคิดหลักของบทเรียนชุดนี้คือ Memory Hierarchy หรือ “ลำดับชั้นหน่วยความจำ” ซึ่งจัดหน่วยความจำหลายชนิดตามความเร็ว ขนาด และราคา เพื่อให้ระบบดูเหมือนมีทั้งความเร็วสูงและความจุสูงในราคาที่เหมาะสม

Memory System: Speed, Size, and Cost



รูปจากสไลด์: Memory System: Speed, Size, and Cost (Lecture 9_MemorySystem, page 5)

จำง่าย: ยิ่งอยู่ใกล้ CPU ยิ่งเร็วแต่แพงและเล็ก เช่น Register และ Cache; ยิ่งอยู่ไกล CPU ยิ่งช้าแต่ใหญ่และถูก เช่น Main Memory และ Disk

เส้นทางภาพรวมคือ Register → Cache → Main Memory → Secondary Storage/Disk ส่วน Virtual Memory

ทำให้โปรแกรมเห็นพื้นที่หน่วยความจำเสมือนที่ใหญ่และเป็นของตัวเอง โดยระบบปฏิบัติการและฮาร์ดแวร์ช่วยแปลงที่อยู่ให้

บทที่ 1: ภาพรวมระบบหน่วยความจำ (Memory System Overview)

1.1 Memory System คืออะไร

ระบบหน่วยความจำ (Memory System) คือส่วนของคอมพิวเตอร์ที่เก็บคำสั่ง (instructions) และข้อมูล (data) ที่โปรแกรมต้องใช้ระหว่างการทำงาน โดยรวมตั้งแต่ Register ใน CPU, Cache, Main Memory ไปจนถึง Disk

ใน Lecture 9 ระบุว่า Main memory เก็บโปรแกรมที่กำลัง execute และข้อมูลที่โปรแกรมใช้งาน ขณะที่เมื่อเปิดเครื่อง executable files จะถูกเก็บใน secondary storage เช่น disk

English summary: The memory system stores executing programs and their data, using multiple storage levels to balance speed, capacity, and cost.

1.2 ทำไม Memory มีผลต่อความเร็ว

คำสั่งทุกคำสั่งต้องถูก fetch และข้อมูลจำนวนมากต้องถูก load/store ถ้า CPU ต้องรอ main memory บ่อย โปรแกรมจะช้าลงอย่างมาก ปัญหานี้เรียกว่า memory bottleneck

ออกสอบบ่อย: ข้อสอบมักถามว่า “ทำไมต้องมี Cache?” คำตอบคือเพราะ Main Memory ช้ากว่า CPU มาก จึงใช้ Cache เป็นหน่วยความจำเล็กแต่เร็วเพื่อเก็บส่วนที่ใช้งานบ่อย

1.3 Memory Hierarchy

ระดับ	ตัวอย่าง	ความเร็ว	ขนาด	ราคา/บิต	หน้าที่
ใกล้ CPU มากที่สุด	Register	เร็วที่สุด	เล็กมาก	แพงมาก	เก็บ operands/ผลลัพธ์ชั่วคราว
กลางบน	Cache (L1/L2/L3)	เร็วมาก	เล็ก-กลาง	แพง	เก็บข้อมูล/คำสั่งที่ใช้บ่อย
กลาง	Main Memory / RAM	ปานกลาง	ใหญ่กว่า cache	ถูกกว่า	เก็บโปรแกรมและข้อมูลขณะรัน
ล่าง	SSD/HDD	ช้าที่สุดในกลุ่มนี้	ใหญ่มาก	ถูกมาก	เก็บไฟล์ถาวรและใช้เป็น backing store ของ virtual memory

ระวังสับสน: Memory Hierarchy ไม่ได้แปลว่ามี memory แค่ชนิดเดียว แต่คือการจัดหลายชนิดให้ทำงานร่วมกันเพื่อหลอกให้ CPU “รู้สึก” ว่ามีหน่วยความจำที่ทั้งเร็วและใหญ่

1.4 Speed, Size, Cost per bit

หลัก trade-off คือ fast memory มีราคาแพงและมีขนาดเล็ก ส่วน large memory มักช้ากว่าและราคาต่อบิตถูกกว่า การออกแบบจึงไม่ได้ใช้ memory ชนิดเดียวทั้งหมด แต่ใช้หลายชั้นประกอบกัน

English summary: Memory hierarchy exploits the trade-off between speed, size, and cost per bit.

บทที่ 2: Main Memory และการเชื่อมต่อกับ CPU

2.1 Main Memory และ Word Size

หน่วยความจำหลัก (Main Memory) คือหน่วยความจำที่ CPU เข้าถึงเพื่ออ่านคำสั่งและข้อมูลขณะโปรแกรมทำงาน

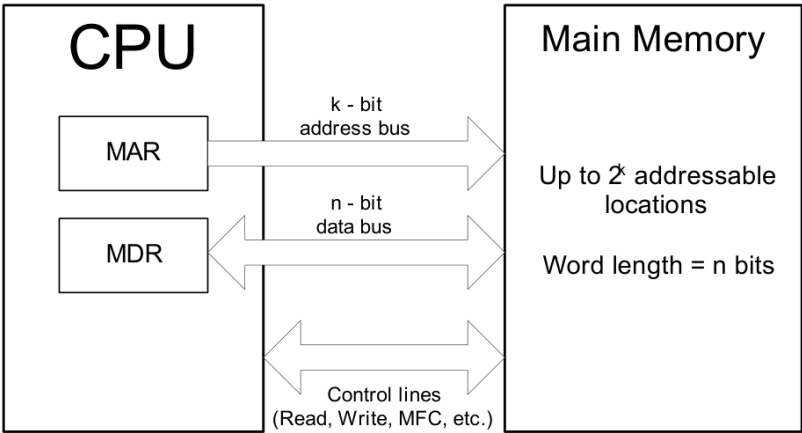
ข้อมูลถูกจัดเก็บและเรียกคืนเป็นหน่วยที่เรียกว่า word

ขนาดคำ (Word size) คือจำนวนบิตที่ CPU ประมวลผลเป็นหน่วยหลัก เช่น 16-bit, 32-bit หรือ 64-bit โดย data bus size, instruction size และ address size มักเกี่ยวข้องกับ word size

English summary: A word is the natural unit of data handled by the CPU and memory system.

2.2 Address bus, Data bus, Control lines

Connection Between Memory and CPU



รูปจากสไลด์: Connection Between Memory and CPU (Lecture 9_MemorySystem, page 9)

องค์ประกอบ	หน้าที่	ตัวอย่าง
Address bus	ส่งที่อยู่ของตำแหน่งหน่วยความจำที่ต้องการเข้าถึง	k-bit address bus เลือกได้ 2^k locations
Data bus	ส่งข้อมูลระหว่าง CPU กับ memory	n-bit data bus ส่งได้ n bits ต่อรอบ
Control lines	ควบคุมชนิดของการทำงานและสัญญาณสถานะ	Read, Write, MFC
MAR (Memory Address Register)	เก็บ address ที่ CPU ต้องการเข้าถึง	ถ้าจะอ่านที่อยู่ 0x1000 ต้องใส่ใน MAR
MDR (Memory Data Register)	เก็บข้อมูลที่อ่านจาก memory หรือข้อมูลที่จะเขียนไป memory	ข้อมูล 32-bit ที่อ่านได้จะมาอยู่ใน MDR

2.3 ขั้นตอน Read from Memory

14. CPU ใส่ address ที่ต้องการลงใน MAR
15. CPU ตั้งค่าสัญญาณ Read หรือ $R/W = 1$
16. Memory นำข้อมูลจาก address นั้นขึ้น data bus
17. Memory ส่งสัญญาณ MFC (Memory Function Completed)
18. CPU รับข้อมูลจาก data bus เข้า MDR

2.4 ขั้นตอน Write to Memory

19. CPU ใส่ address ลงใน MAR
20. CPU ใส่ข้อมูลที่จะเขียนลงใน MDR
21. CPU ตั้งค่าสัญญาณ Write หรือ $R/W = 0$
22. Memory รับ address และ data จาก bus แล้วเขียนลงตำแหน่งที่เลือก

2.5 Memory Access Time และ Memory Cycle Time

Memory Access Time คือเวลาตั้งแต่เริ่มคำสั่งเข้าถึง memory จนได้ผลลัพธ์เสร็จ ส่วน Memory Cycle Time คือเวลาขั้นต่ำระหว่างการเริ่ม memory operation สองครั้งติดกัน

ระวังสับสน: Access time คือเวลาของ operation หนึ่งครั้ง แต่ cycle time คือจังหวะที่ memory พร้อมเริ่ม operation ถัดไป บางระบบ cycle time อาจมากกว่า access time

2.6 Memory Interleaving

Memory Interleaving คือการแบ่ง main memory เป็นหลาย module เพื่อให้เข้าถึงข้อมูลต่อเนื่องได้แบบทับซ้อนกัน ช่วยลดเวลารอคอยโดยเฉพาะตอน cache miss ที่ต้องดึง block หลายคำจาก main memory

2.7 ตัวอย่างคำนวณ Addressable Locations

สูตรพื้นฐานจำนวนตำแหน่งที่ address ได้ $= 2^k$ locations

ถ้า address bus มี 16 บิต จะ address ได้ $2^{16} = 65,536$ locations หรือ 64K locations ถ้าแต่ละ location มี 1 byte จะเท่ากับ 64 KB

ถ้าเป็น 32-bit address space จะมี 2^{32} bytes = 4 GB โดยประมาณ

English summary: With k address bits, memory can address 2^k locations.

บทที่ 3: RAM และ ROM

3.1 RAM คืออะไร

หน่วยความจำเข้าถึงแบบสุ่ม (Random Access Memory: RAM) คือหน่วยความจำที่ตำแหน่งใด ๆ

ถูกเข้าถึงด้วยรูปแบบเดียวกันและใช้เวลาคงที่โดยไม่ขึ้นกับตำแหน่ง address

คำว่า random ใน RAM ไม่ได้แปลว่าสุ่มมั่ว แต่หมายถึง “เลือกตำแหน่งใดก็ได้โดยตรง” ต่างจาก tape ที่ต้องไล่ตามลำดับ

English summary: RAM allows direct access to any location with roughly uniform access time.

3.2 ROM คืออะไร

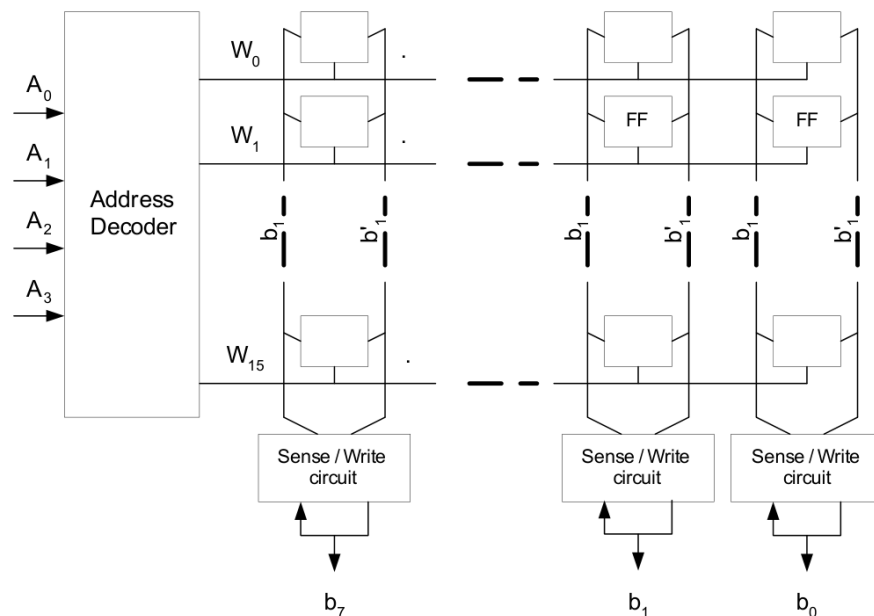
หน่วยความจำอ่านอย่างเดียว (Read Only Memory: ROM) ใช้เก็บโปรแกรมหรือข้อมูลที่คงที่ เช่น firmware หรือ boot code ROM เป็น non-volatile memory คือไฟดับแล้วยังเก็บข้อมูลอยู่

ชนิด	Volatile?	เขียนได้ง่ายไหม	ใช้ทำอะไร
RAM	ใช่ ข้อมูลหายเมื่อปิดไฟ	เขียน/อ่านได้เร็ว	หน่วยความจำหลักระหว่างรันโปรแกรม
ROM	ไม่ใช่ ข้อมูลยังอยู่	เขียนต้องใช้กระบวนการพิเศษ	Firmware, boot code

3.3 RAM Architecture

RAM Architecture

16 byte Address
หรือ 16x8 bit Address



รูปจากสไลด์: RAM Architecture (Lecture 9_MemorySystem, page 20)

RAM chip จัด memory cells เป็น array แต่ละ cell เก็บ 1 bit แลวเดียวกันต่อกับ word line ส่วนคอลัมน์ต่อกับ bit line และวงจร sense/write circuit จะอ่านหรือเขียนค่าจาก bit line

ส่วนประกอบ	คำอธิบาย
Memory cell	วงจรเล็ก ๆ ที่เก็บข้อมูล 1 bit
Word line	เส้นที่เลือกแถวของ memory cells
Bit line	เส้นที่รับส่งข้อมูลของคอลัมน์
Address decoder	แปลง address เป็นการเลือก word line ที่ถูกต้อง
Sense/write circuit	อ่านค่า bit line หรือเขียนค่าใหม่ลง cell

3.4 SRAM (Static RAM)

Static RAM (SRAM)

- Each RAM cell consists of 2 inverters cross-connected to form a latch.
- A latch is connected to bit lines through transistors.
- If x is 1 and y is 0, the cell state is 1.
- As long as a signal on the word line is grounded, the cell will retain the state.
- When the power is interrupted, states in SRAM will be lost.

รูปจากสไลด์: SRAM Circuit (Lecture 9_MemorySystem, page 25)

SRAM ใช้ latch ที่สร้างจาก inverter cross-connected เพื่อเก็บค่า bit ข้อดีคือเร็วและไม่ต้อง refresh ตลอดเวลาที่มีไฟเลี้ยง แต่ใช้ transistor หลายตัวต่อ cell จึงแพงและกินพื้นที่

English summary: SRAM stores bits using latch circuits, making it fast but expensive and less dense.

3.5 DRAM (Dynamic RAM)

(Asynchronous) Dynamic RAM (DRAM)

- SRAM is fast, but expensive with several transistors. DRAM circuit is simpler. However, it **cannot retain** the state indefinitely.
- Information stored in DRAM is in the form of a charge on a capacitor. The contents must be **periodically recharged** (The charge can only be held for some 10 milliseconds).
- To store information in the cell, a transistor T is turned on, which causes a known amount of charge to be stored in the capacitor.

ตัว charge

รูปจากสไลด์: DRAM Circuit (Lecture 9_MemorySystem, page 28)

DRAM ใช้ capacitor เก็บประจุแทนค่า bit และใช้ transistor เป็นสวิตช์ ข้อดีคือเก็บข้อมูลได้หนาแน่นและราคาถูกกว่า SRAM แต่ประจุรั่ว จึงต้อง refresh เป็นระยะ

English summary: DRAM stores bits as charge in capacitors, making it dense and cheap but requiring refresh.

3.6 เปรียบเทียบ SRAM vs DRAM

ประเด็น	SRAM	DRAM
โครงสร้าง cell	Latch จาก inverter + transistor	Capacitor + transistor
ความเร็ว	เร็วกว่า	ช้ากว่า
ต้อง refresh?	ไม่ต้อง	ต้อง refresh
ความหนาแน่น	ต่ำกว่า	สูงกว่า
ราคา/บิต	แพงกว่า	ถูกกว่า
ใช้ที่ไหน	Cache	Main memory

3.7 ตัวอย่าง RAM Organization

หน่วยความจำ 1K bit สามารถจัดได้หลายแบบ เช่น 128×8 หมายถึงมี 128 words แต่ละ word ยาว 8 bits รวม 1024 bits ส่วน 1024×1 หมายถึงมี 1024 words แต่ละ word ยาว 1 bit รวม 1024 bits เช่นกัน

จำง่าย: รูปแบบ $A \times B$ แปลว่า มี A words และแต่ละ word มี B bits ดังนั้นความจุรวม = $A \times B$ bits

บทที่ 4: Cache Memory

4.1 Cache Memory คืออะไร

แคช (Cache) คือหน่วยความจำขนาดเล็กแต่เร็วมาก อยู่ระหว่าง CPU กับ Main Memory จุดประสงค์คือทำให้ CPU รู้สึกเหมือนว่า Main Memory เร็วกว่าความเป็นจริง

English summary: Cache is a small and fast memory placed between the CPU and main memory to reduce average memory access time.

4.2 Locality of Reference

หลักการที่ทำให้ cache ใช้ได้ผลคือ Locality of Reference คือโปรแกรมมักไม่ได้เข้าถึงข้อมูลกระจายเท่ากันทั้งหมด แต่มีแนวโน้มใช้ข้อมูล/คำสั่งบางส่วนซ้ำหรือใกล้กัน

Temporal Locality

Temporal locality คือถ้าข้อมูลหรือคำสั่งหนึ่งเพิ่งถูกใช้ มีโอกาสสูงว่าจะถูกใช้อีกในอนาคตอันใกล้ ตัวอย่างคือ loop ที่ execute instruction เดิมซ้ำหลายรอบ

English summary: Temporal locality means recently accessed items are likely to be accessed again soon.

Spatial Locality

Spatial locality คือถ้าข้อมูลหรือคำสั่งหนึ่งถูกใช้ ตำแหน่งใกล้ ๆ กันมักถูกใช้ตามมา เช่น array traversal หรือ instruction sequence ที่อยู่ติดกัน

English summary: Spatial locality means nearby addresses are likely to be accessed soon.

4.3 Cache Hit และ Cache Miss

Cache hit คือข้อมูลที่ CPU ต้องการอยู่ใน cache จึงอ่าน/เขียนได้เร็ว ส่วน cache miss คือข้อมูลไม่อยู่ใน cache ต้องไปดึงจาก main memory ซึ่งเสียเวลามากกว่า

กรณี	ความหมาย	ผลกระทบ
Read hit	อ่านแล้วเจอใน cache	CPU อ่านจาก cache ได้ทันที
Read miss	อ่านแล้วไม่เจอใน cache	ต้องโหลด block จาก main memory เข้ามา cache
Write hit	เขียนแล้วตำแหน่งอยู่ใน cache	ใช้ write-through หรือ write-back
Write miss	เขียนแล้วไม่อยู่ใน cache	ขึ้นกับ policy เช่น write-through อาจเขียน main memory โดยตรง

4.4 Write-through vs Write-back

Policy	ทำงานอย่างไร	ข้อดี	ข้อเสีย
Write-through	อัปเดต cache และ main memory	ง่าย memory มีข้อมูลล่าสุดเสมอ	เขียน main memory บ่อย อาจช้า

	พร้อมกัน		
Write-back	อัปเดต cache ก่อน ตั้ง dirty/modified bit แล้วค่อยเขียนกลับเมื่อ block ถูกแทนที่	ลดจำนวน write ไป main memory	วงจรซับซ้อน ต้องจัดการ dirty bit

ออกสอบบ่อย: Dirty bit / Modified bit ใช้กับ write-back เพื่อบอกว่า block ใน cache ถูกแก้แล้วแต่ main memory ยังไม่อัปเดต

4.5 Load-through และ Pipeline Stall

เมื่อเกิด read miss ระบบอาจโหลดทั้ง block เข้า cache ก่อนแล้วส่ง word ที่ต้องการให้ CPU หรือใช้ load-through protocol คือส่ง word ให้ CPU ทันทีที่อ่านได้จาก main memory เพื่อลดเวลารอ แต่ hardware ซับซ้อนขึ้น

ใน processor แบบ pipeline, cache miss อาจทำให้ instruction execution stall เพราะขั้นตอน fetch และ data access แย่ง memory กัน วิธีแก้ที่ง่ายคือแยก instruction cache (I-cache) และ data cache (D-cache) เพื่อให้ fetch instruction และ access data ทำพร้อมกันได้

บทที่ 5: Cache Mapping

5.1 Block, Tag, Word, Set

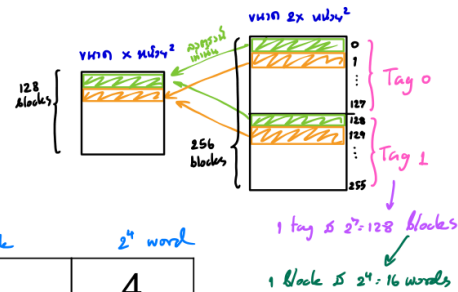
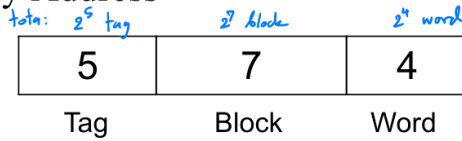
Block คือกลุ่มของ address ที่ติดกันและถูกย้ายระหว่าง main memory กับ cache เป็นหน่วยเดียว การแบ่ง address จะมี field เพื่อเลือกตำแหน่งใน cache และตรวจสอบว่า block ที่อยู่ใน cache คือ block ที่ต้องการจริงหรือไม่

คำศัพท์	ความหมาย
Block	กลุ่มคำ/byte ต่อเนื่องที่ย้ายเข้า cache พร้อมกัน
Word / Offset	ตำแหน่งย่อยภายใน block
Tag	ส่วนของ address ที่ยืนยันว่า cache line นี้มาจาก main memory block ไດ
Index / Block field	ส่วนที่เลือก cache block/line ใน direct mapping
Set	กลุ่มของ cache lines ใน set-associative cache
Valid bit	บอกว่า cache line มีข้อมูลที่ใช้ได้หรือไม่

5.2 Direct Mapping

Direct Mapping (1)

- Memory Address



- 'Tag' is used to identify which block from a memory is currently in a cache.
- 'Block' is used to select a block in a cache.
- 'Word' is used to select a word in a block.

รูปจากสไลด์: Direct mapping address fields (Lecture 10_CacheMemory, page 11)

Direct Mapping กำหนดว่า main memory block j จะถูกวางใน cache block หมายเลข j modulo จำนวน block ใน cache เท่านั้น จึง hardware ง่ายและเร็ว แต่ไม่ยืดหยุ่น เพราะหลาย memory blocks อาจชนกันใน cache block เดียว

$$\text{สูตร cache block} = \text{memory block number} \bmod \text{number_of_cache_blocks}$$

ระวังสับสน: Direct mapping อาจเกิด collision แม้ cache ยังไม่เต็ม เพราะ block หลายตัวถูกบังคับให้ลงตำแหน่งเดียวกัน

5.3 Fully Associative Mapping

Associative หรือ Fully Associative Mapping อนุญาตให้ main memory block ใด ๆ วางใน cache block ใดก็ได้ จึงยืดหยุ่นมาก ลด collision แต่ต้องเปรียบเทียบ tag กับหลาย cache lines ทำให้ hardware ซับซ้อนและอาจช้ากว่า

5.4 Set-Associative Mapping

Set-Associative Mapping (1)

- Memory Address

6	6	4
Tag	Set	Word

- ‘Tag’ is used to identify which block from a memory is currently in a cache.
- ‘Set’ is used to select a set in a cache.
- ‘Word’ is used to select a word in a block.

รูปจากสไลด์: Set-associative mapping address fields (Lecture 10_CacheMemory, page 17)

Set-Associative Mapping เป็นทางสายกลาง โดยแบ่ง cache เป็นหลาย set แล้ว memory block j จะ map ไปยัง $\text{set} = j \text{ modulo จำนวน set ภายใน set}$ นั้นสามารถวางใน line ใดก็ได้ ถ้า 1 set มี k blocks เรียกว่า k -way set associative cache

สูตร $\text{cache set} = \text{memory block number} \bmod \text{number_of_sets}$

Mapping	ตำแหน่งที่ block ลงได้	ข้อดี	ข้อเสีย
Direct	ตำแหน่งเดียว	เร็ว hardware ง่าย	collision สูง ไม่ยืดหยุ่น
Fully Associative	ตำแหน่งใดก็ได้ใน cache	ยืดหยุ่นมาก	ค้นหา tag แพง/ซับซ้อน
Set-Associative	set ที่กำหนด แต่เลือก line ใน set ได้	สมดุลระหว่างเร็วและยืดหยุ่น	ซับซ้อนกว่า direct

5.5 ตัวอย่างแบ่ง Address

ตัวอย่างจาก Lecture 10: main memory ใช้ address 16 bit มี 64K words และมองเป็น 4K blocks ถ้า block ละ 16 words จะต้องใช้ word field = $\log_2(16)=4$ bits

Mapping	จำนวน bits ตามตัวอย่าง	เหตุผล
Direct mapping	Tag 5 bits / Block 7 bits / Word 4 bits	cache มี 128 blocks $\rightarrow$ block field 7 bits, address รวม 16 bits
Fully associative	Tag 12 bits / Word 4 bits	ไม่มี index/block field เพราะ block ลงได้ทุกที่

2-way set associative	Tag 6 bits / Set 6 bits / Word 4 bits	cache 128 blocks และ 2-way → 64 sets → set field 6 bits
-----------------------	---------------------------------------	--

5.6 Replacement Algorithm

เมื่อ cache ที่เลือกได้หลายตำแหน่งเต็ม ต้องเลือก block เก่าที่จะถูกแทนที่ เรียกว่า replacement algorithm

Algorithm	แนวคิด	เหมาะเมื่อ
LRU (Least Recently Used)	ทิ้ง block ที่ไม่ได้ใช้นานที่สุด	ต้องการใช้ประโยชน์จาก temporal locality
Random Selection	สุ่มเลือก block ทิ้ง	ต้องการ hardware ง่าย
Remove oldest block	ทิ้ง block ที่อยู่มานานที่สุด	ง่ายกว่า LRU แต่ไม่ดูการใช้งานล่าสุด

ออกสอบบ่อย: LRU ไม่ได้เลือก block ที่เก่าที่สุดตามเวลาเข้ามาเสมอ แต่เลือก block ที่ “ไม่ได้ถูกอ้างอิงนานที่สุด”

บทที่ 6: Cache Performance

6.1 Hit Rate, Miss Rate, Miss Penalty

Hit rate (h) คือสัดส่วนของ memory accesses ที่เจอข้อมูลใน cache ส่วน miss rate คือ 1-h และ miss penalty คือเวลาที่เสียเพิ่มเมื่อไม่เจอใน cache ต้องไปดึงข้อมูลจาก memory ที่ช้ากว่า

Hit Rate and Miss Penalty (2)

t_{ave} is the average time to access information

$$t_{ave} = ht_C + (1-h)t_M$$

h is a hit rate

t_C is an access time of the cache

t_M is a miss penalty (an access time of the mem?)

รูปจากสไลด์: Average memory access time formula (Lecture 10_CacheMemory, page 26)

$$\text{สูตรสำคัญ } t_{avg} = h \times t_C + (1 - h) \times t_M$$

ตัวแปร	ความหมาย
t_avg	เวลาเฉลี่ยในการเข้าถึงข้อมูล
h	hit rate
t_C	เวลาเข้าถึง cache
t_M	เวลาของ miss path เช่น miss penalty หรือเวลาเข้าถึง main memory ตามนิยามโจทย์

ระวังสับสน: บางตำราเขียน $AMAT = \text{hit time} + \text{miss rate} \times \text{miss penalty}$ แต่สไลด์ใช้ $t_{avg} = h t_C + (1-h) t_M$ ดังนั้นทำข้อสอบต้องดูนิยาม t_M ในโจทย์ให้ชัด

6.2 ตัวอย่างคำนวณ 1

กำหนด $h = 0.9$, $t_C = 1$ cycle, $t_M = 10$ cycles

23. แทนสูตร $t_{avg} = 0.9 \times 1 + (1-0.9) \times 10$

24. ได้ $t_{avg} = 0.9 + 1 = 1.9$ cycles

6.3 ตัวอย่างคำนวณ 2

กำหนด $h = 0.98$, $t_C = 2$ ns, $t_M = 80$ ns

25. miss rate = $1-0.98 = 0.02$

26. $t_{avg} = 0.98 \times 2 + 0.02 \times 80 = 1.96 + 1.6 = 3.56$ ns

6.4 ตัวอย่างคำนวณ 3: เปรียบเทียบสอง cache

Cache A: $h=0.95$, $t_C=1$ ns, $t_M=50$ ns. Cache B: $h=0.98$, $t_C=3$ ns, $t_M=50$ ns

27. A: $t_{avg} = 0.95 \times 1 + 0.05 \times 50 = 3.45$ ns

28. B: $t_{avg} = 0.98 \times 3 + 0.02 \times 50 = 3.94$ ns

29. แม้ B hit rate สูงกว่า แต่ช้ากว่า เพราะ hit time สูงกว่า

จำง่าย: Hit rate สูงไม่ได้แปลว่าเร็วเสมอ ต้องดู t_C และ miss penalty ด้วย

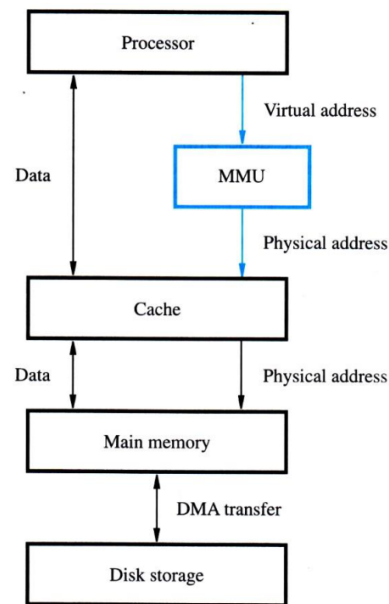
บทที่ 7: Virtual Memory

7.1 ปัญหาที่ Virtual Memory แก้

Virtual Memory เข้ามาแก้ปัญหาหลักหลายอย่าง ได้แก่ RAM ไม่พอ, การต้องจองหน่วยความจำแบบต่อเนื่อง, fragmentation, โปรแกรมเขียนทับกัน และการแยกพื้นที่ของแต่ละโปรแกรมเพื่อความปลอดภัย

Logical and Physical Address

- Logical address, LA
 - or Virtual Address, VA
 - is the address that a program uses to refer a memory location.
- Physical address, PA
 - is the actual address.
- Memory Management Unit (MMU)
 - handles a virtual memory system. The MMU knows how to map from LA to PA or from PA to LA.



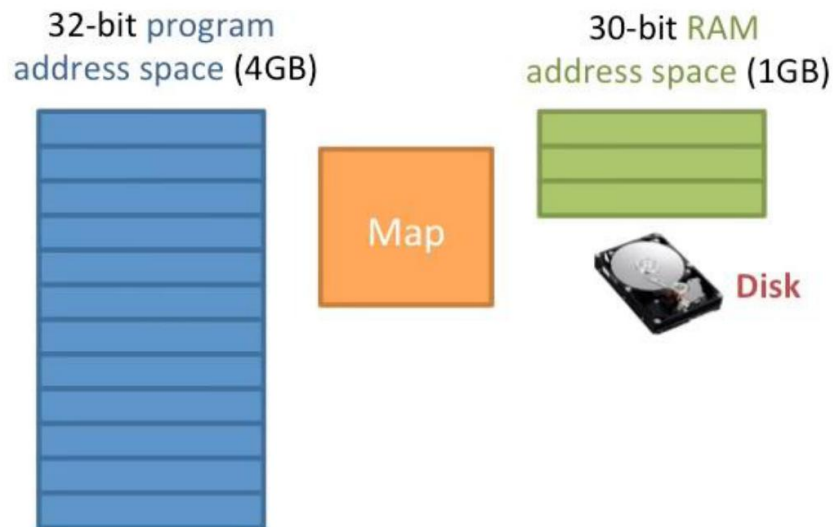
รูปจากสไลด์: Logical/Physical Address and MMU (Lecture 11_VirtualMemory, page 7)

ปัญหา	คำอธิบาย	Virtual Memory ช่วยอย่างไร
Not enough RAM	โปรแกรมอาจมี address space 4GB แต่ RAM จริงมีน้อยกว่า	ให้โปรแกรมเห็น virtual space ใหญ่ แล้ว map เฉพาะส่วนที่ใช้จริงลง RAM/Disk
Contiguous allocation	โปรแกรมใหญ่ต้องการพื้นที่ต่อเนื่อง ทำให้จัดวางยาก	แบ่งเป็น page จึงวาง physical frames ไม่ต่อเนื่องได้
Fragmentation	หน่วยความจำแตกเป็นช่องว่างเล็ก ๆ	paging ลดปัญหาการต้องหา block ต่อเนื่องใหญ่
Writing over each other	สองโปรแกรมใช้ address เหมือนกันแต่ไม่ควรชนกัน	แต่ละโปรแกรมมี virtual address space ของตัวเอง
Isolation	ป้องกันโปรแกรมหนึ่งเข้าถึง memory ของอีกโปรแกรม	MMU/OS คุม mapping และ permission

7.2 Virtual Memory คืออะไร

หน่วยความจำเสมือน (Virtual Memory) คือเทคนิคที่ทำให้แต่ละโปรแกรมเห็นพื้นที่ address ของตัวเองเหมือนมีหน่วยความจำขนาดใหญ่ต่อเนื่อง แต่จริง ๆ แล้ว address นั้นถูก map ไปยัง physical memory และบางส่วนอาจอยู่บน disk

Understand GIVE&MAP



รูปจากสไลด์: GIVE & MAP idea (Lecture 11_VirtualMemory, page 8)

English summary: Virtual memory gives each program its own virtual address space and maps it to physical memory and disk.

7.3 Logical/Virtual Address และ Physical Address

Logical Address หรือ Virtual Address (LA/VA) คือ address ที่โปรแกรมใช้ ส่วน Physical Address (PA) คือ address จริงใน RAM การแปลง LA เป็น PA เรียกว่า address translation หรือ address binding

$$\text{แนวคิดฟังก์ชัน} PA = f(LA)$$

ตัวที่ทำงานสำคัญคือ Memory Management Unit (MMU) ซึ่งรู้วิธีแปลง address และตรวจสอบสิทธิ์การเข้าถึง

7.4 Disk ใน Virtual Memory

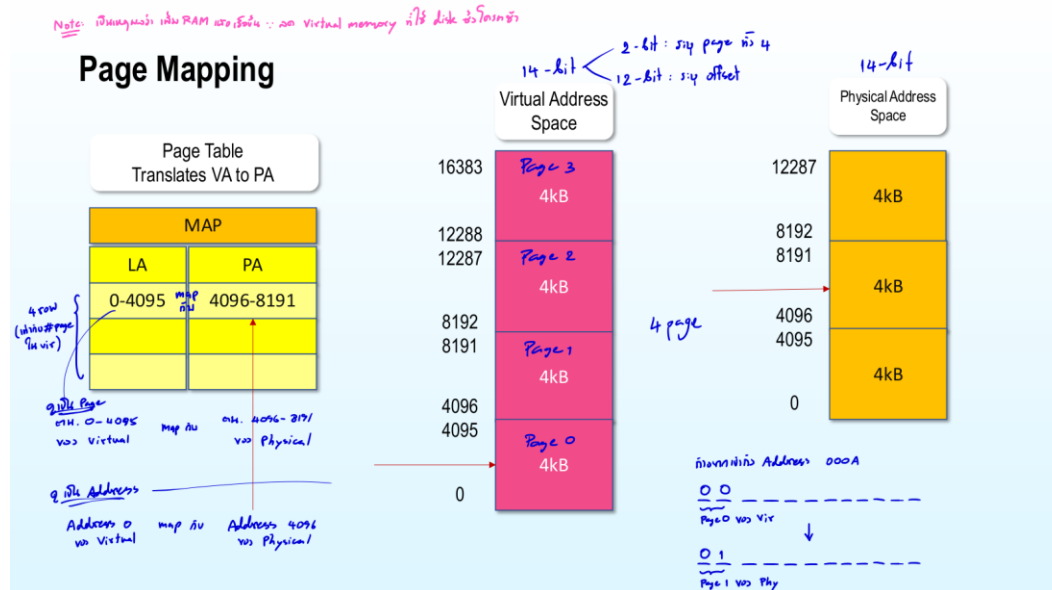
Disk ทำหน้าที่เป็น backing store สำหรับข้อมูล/หน้า memory ที่ยังไม่อยู่ใน RAM หรือถูกย้ายออกจาก RAM ชั่วคราว เมื่อโปรแกรมเข้าถึงหน้าที่ไม่มีใน RAM จะเกิด page fault แล้ว OS จะโหลดจาก disk เข้ามา

จำง่าย: Virtual Memory = GIVE & MAP: “ให้” โปรแกรมเห็นพื้นที่ใหญ่และเป็นของตัวเอง แล้ว “map” ไปยัง RAM จริงและ Disk

บทที่ 8: Paging และ Page Table

8.1 Page, Page size, Frame, Offset

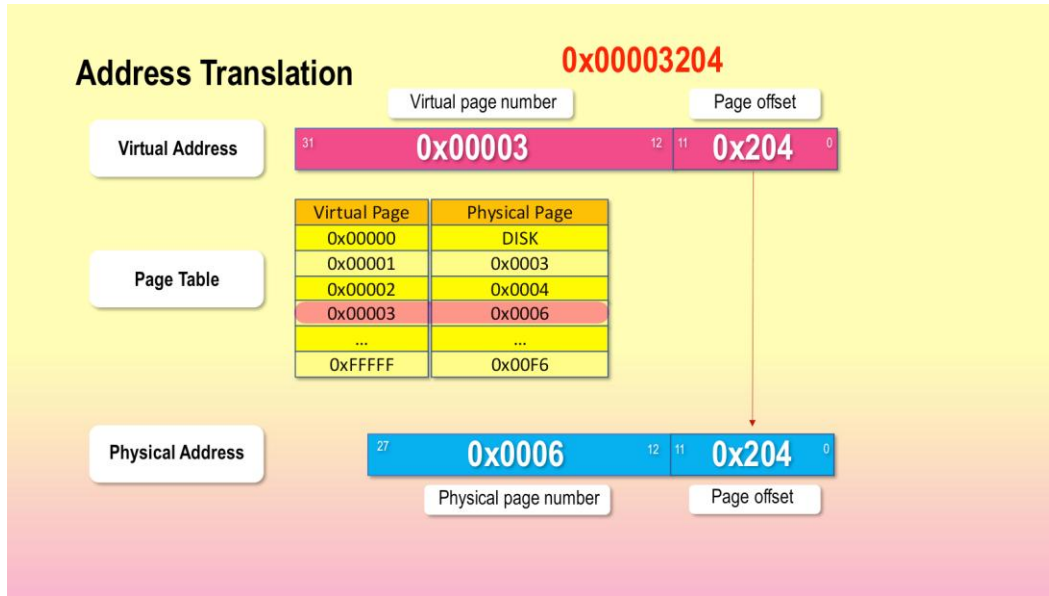
Paging คือการแบ่ง virtual address space เป็นชิ้นขนาดเท่ากันเรียกว่า page และแบ่ง physical memory เป็นชิ้นขนาดเท่ากันเรียกว่า frame หรือ page frame



รูปจากสไลด์: Page mapping between virtual address space and physical memory (Lecture 11_extended, page 10)

คำศัพท์	ความหมาย
Page	ชิ้นส่วนของ virtual address space
Page size	ขนาดของ page เช่น 4 KB, 8 KB, 2 MB
Page number	หมายเลข page ใน virtual address
Offset	ตำแหน่งภายใน page
Frame / Page frame	ช่องใน physical memory ที่ page ถูกโหลดเข้าไป
Page table	ตารางที่ map page number → frame number หรือบอกว่าอยู่ disk

8.2 การแบ่ง Address เป็น Page Number และ Offset



รูปจากสไลด์: Address Translation: page number and offset (Lecture 11_extended, page 11)

ถ้า page size = 4 KB = 2^{12} bytes จะใช้ offset 12 bits เพราะต้องเลือก byte ภายใน page ส่วน bits ที่เหลือเป็น virtual page number

สูตร offset bits = $\log_2(\text{page size in bytes})$

ออกสอบบ่อย: ถ้าหน้า page มีขนาดเป็น 2^n bytes ค่า offset ต้องใช้ n bits เสมอ

8.3 Page Table Entry

Page Table Entry (PTE) โดยทั่วไปประกอบด้วย frame number หรือ disk location, valid/present bit, protection bits, dirty bit และ referenced/use bit ขึ้นกับสถาปัตยกรรม

Field	หน้าที่
Frame number	บอกว่า page นี้อยู่ frame ไหนใน RAM
Valid/Present bit	บอกว่า page อยู่ใน RAM และ mapping ใช้ได้หรือไม่
Protection bits	สิทธิ์ read/write/execute
Dirty bit	page ถูกแก้ไขแล้วหรือยัง
Referenced bit	เคยถูกใช้งานหรือไม่ ใช้ช่วย replacement

8.4 Page Fault

Page fault คือเหตุการณ์ที่โปรแกรมเข้าถึง virtual page ที่ไม่อยู่ใน physical memory หรือ invalid ทำให้ CPU/OS ต้องหยุดการทำงานชั่วคราวเพื่อโหลด page จาก disk เข้ามา

30. CPU สร้าง virtual address
31. MMU ตรวจ page table แล้วพบ present bit = 0
32. เกิด trap ไปยัง operating system

33. OS หา page ใน disk/backing store
34. หา free frame หรือเลือก victim frame ด้วย replacement algorithm
35. ถ้า victim dirty ต้องเขียนกลับ disk
36. โหลด page ที่ต้องการเข้า frame
37. อัปเดต page table แล้ว restart instruction

8.5 Demand Paging

Demand Paging คือการโหลด page เข้ามาใน RAM เฉพาะเมื่อถูกใช้งานจริง ไม่โหลดทั้งโปรแกรมเข้ามาก่อนทั้งหมด วิธีนี้ประหยัด RAM แต่ถ้า page fault บ่อยจะช้ามาก

8.6 ตัวอย่าง Address Translation

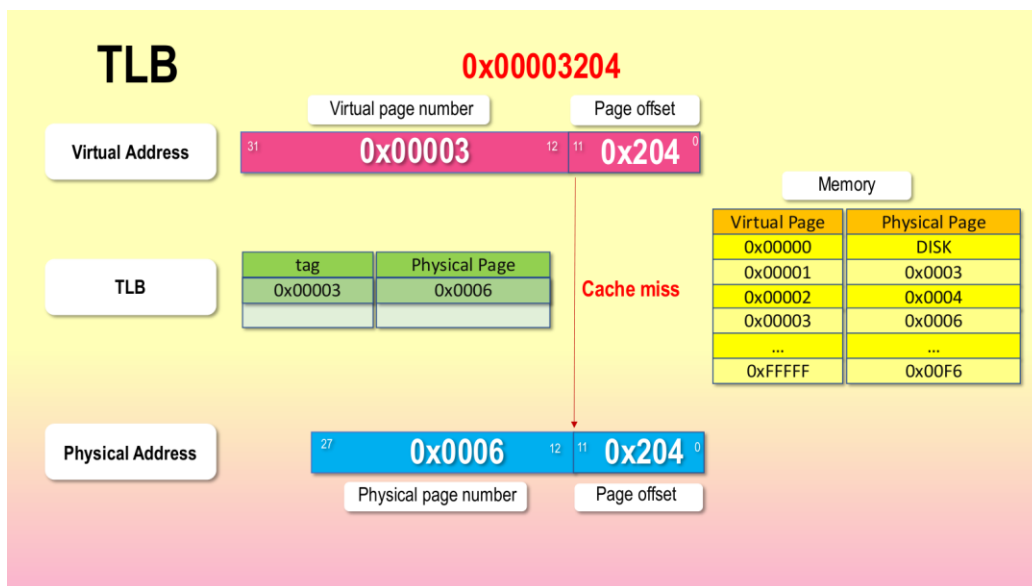
ตัวอย่างเสริม: ระบบมี virtual address 16 bits, page size = 1 KB = 2^{10} bytes ดังนั้น offset = 10 bits และ page number = 6 bits

ให้ VA = 0x1234 = 4660 decimal

38. page number = $\text{floor}(4660 / 1024) = 4$
39. offset = $4660 \bmod 1024 = 564$
40. สมมติ page table บอก page 4 $\rightarrow$ frame 9
41. physical address = $\text{frame} \times \text{page_size} + \text{offset} = 9 \times 1024 + 564 = 9780 \text{ decimal} = 0x2634$

English summary: Address translation keeps the offset unchanged and replaces the virtual page number with a physical frame number.

8.7 TLB: Translation Lookaside Buffer



รูปจากสไลด์: TLB: cache for address translation (Lecture 11_extended, page 12)

TLB (Translation Lookaside Buffer) คือ cache เฉพาะงานสำหรับเก็บผลการแปลง address ล่าสุด ถ้า TLB hit จะไม่ต้องอ่าน page table ใน RAM จึงลดค่าใช้จ่ายของ address translation ได้มาก

บทที่ 9: เชื่อม Cache กับ Virtual Memory

9.1 Cache กับ Virtual Memory ต่างกันอย่างไร

Cache และ Virtual Memory ต่างเป็นเทคนิคที่ใช้แนวคิด locality และ mapping แต่แก้ปัญหาคนละแบบ Cache แก้ปัญหาความเร็วระหว่าง CPU กับ Main Memory ส่วน Virtual Memory แก้ปัญหาความจุ การจัดการ address และ isolation ระหว่างโปรแกรม

ประเด็น	Cache Memory	Virtual Memory
ปัญหาหลักที่แก้	Speed gap ระหว่าง CPU กับ RAM	RAM ไม่พอ, isolation, address management
หน่วยข้อมูล	Block / Cache line	Page
อยู่ระหว่าง	CPU ↔ Main Memory	Program virtual space ↔ RAM/Disk
ตัวแปลง/ควบคุม	Cache controller	MMU + OS
เหตุการณ์ผิดพลาด	Cache miss	Page fault
เวลาดังโทษ	miss penalty จาก RAM	page fault penalty จาก disk ซึ่งหนักกว่ามาก
ข้อมูลระบุตัวตน	Tag	Page number / frame number

9.2 Cache Miss vs Page Fault

Cache miss มักแก้โดย hardware cache controller ดึง block จาก main memory ส่วน page fault เกี่ยวข้องกับ OS และ disk จึงใช้เวลานานกว่ามาก Page fault หนึ่งครั้งอาจเสียเวลามากกว่า cache miss หลายพันถึงหลายล้านเท่า ขึ้นกับ storage

จำง่าย: Cache miss = หาไม่เจอใน cache แต่ยังมีอยู่ใน RAM; Page fault = page ที่ต้องการไม่อยู่ใน RAM ต้องพึ่ง OS/Disk

9.3 Memory Hierarchy จากมุมมอง CPU

จากมุมมอง CPU การเข้าถึงข้อมูลเริ่มจาก register และ cache ถ้าไม่เจอจึงไป main memory ถ้า virtual page ไม่อยู่ใน RAM จะไป disk ผ่านกลไก virtual memory ดังนั้น main memory เป็น “ชั้นกลาง” ระหว่าง cache และ disk

บทที่ 10: สรุปใหญ่ก่อนสอบ

10.1 Mind map แบบข้อความ

Memory System

- Hierarchy: Register → Cache → Main Memory → Disk
- Main Memory: word size, address bus, data bus, control lines, MAR/MDR, access time/cycle time
- RAM/ROM: volatile vs non-volatile, SRAM vs DRAM, memory cell/word line/bit line
- Cache: locality, hit/miss, write-through/write-back, dirty bit, load-through, I-cache/D-cache
- Cache Mapping: direct, fully associative, set-associative, tag/index/word, valid bit, replacement/LRU
- Performance: hit rate, miss rate, miss penalty, average memory access time
- Virtual Memory: VA/LA, PA, MMU, address translation, page table, page fault, TLB

10.2 สูตรที่ต้องจำ

สูตร	ใช้เมื่อ
$\text{addressable locations} = 2^k$	หา maximum memory locations จาก address bits
$\text{total capacity} = \text{number of words} \times \text{bits per word}$	วิเคราะห์ RAM organization เช่น 128x8
$\text{cache block} = \text{memory block} \bmod \text{number_of_cache_blocks}$	Direct mapping
$\text{cache set} = \text{memory block} \bmod \text{number_of_sets}$	Set-associative mapping
$\text{word/offset bits} = \log_2(\text{block size or page size})$	แบ่ง address field
$t_{\text{avg}} = h \times t_C + (1-h) \times t_M$	cache performance ตามสไลด์
$\text{physical address} = \text{frame number} \times \text{page size} + \text{offset}$	address translation แบบ paging

10.3 Glossary ไทย-อังกฤษ

ไทย	English	ความหมายสั้น
ระบบหน่วยความจำ	Memory System	กลไกเก็บและส่งข้อมูล/คำสั่งให้ CPU
ลำดับชั้นหน่วยความจำ	Memory Hierarchy	การจัด memory หลายระดับตาม speed/size/cost
หน่วยความจำหลัก	Main Memory	RAM ที่เก็บโปรแกรมและข้อมูลขณะรัน
บัสที่อยู่	Address Bus	เส้นส่ง address จาก CPU ไป memory
บัสข้อมูล	Data Bus	เส้นส่ง data ระหว่าง CPU และ memory
แคช	Cache	memory เล็กเร็วมากระหว่าง CPU กับ RAM
การอ้างอิงเชิงเวลา	Temporal Locality	สิ่งที่เพิ่งใช้มีโอกาสถูกใช้ซ้ำ

การอ้างอิงเชิงพื้นที่	Spatial Locality	ตำแหน่งใกล้เคียงมักถูกใช้ตามมา
การโดนแคช	Cache Hit	ข้อมูลที่ต้องการอยู่ใน cache
พลาดแคช	Cache Miss	ข้อมูลไม่อยู่ใน cache ต้องดึงจาก RAM
บิตสกปรก	Dirty/Modified Bit	บอกว่า cache/page ถูกแก้ไขและไม่เขียนกลับ
หน่วยความจำเสมือน	Virtual Memory	ให้โปรแกรมเห็น address space ของตัวเองและ map ไป RAM/Disk
ที่อยู่ตรรกะ	Logical Address	address ที่โปรแกรมใช้
ที่อยู่กายภาพ	Physical Address	address จริงใน RAM
หน่วยจัดการหน่วยความจำ	MMU	ฮาร์ดแวร์แปลง address และคุมสิทธิ์
หน้าหน่วยความจำ	Page	ชิ้นส่วนของ virtual address space
เฟรม	Frame	ช่องใน physical memory สำหรับเก็บ page
ตารางหน้า	Page Table	ตาราง map page → frame/disk
ความผิดพลาดของหน้า	Page Fault	เข้าถึง page ที่ไม่อยู่ใน RAM
ทีแอลบี	TLB	cache สำหรับผลการแปลง address

10.4 จุดที่มักออกสอบ

- อธิบายเหตุผลของ memory hierarchy
- คำนวณ 2^k addressable locations
- เปรียบเทียบ SRAM/DRAM และ RAM/ROM
- แยกขั้นตอน read/write memory ด้วย MAR/MDR
- แบ่ง address เป็น tag/index/word
- คำนวณ direct/set-associative mapping
- คำนวณ t_{avg} จาก hit rate
- อธิบาย $VA \rightarrow PA$ ด้วย page table
- แยก cache miss กับ page fault

10.5 Checklist ก่อนสอบ

- อธิบายได้ว่า CPU เร็วกว่า memory อย่างไรและทำไม cache ช่วยได้
- รู้ว่า k -bit address bus address ได้ 2^k locations
- แยก SRAM กับ DRAM ได้แบบไม่สลับ
- ทำโจทย์ Tag/Index/Offset ได้
- รู้ความหมาย valid bit และ dirty bit
- จำสูตร t_{avg} และตีความตัวแปรได้
- เข้าใจว่า virtual address ไม่ใช่ physical address
- ทำโจทย์ page number + offset + frame number ได้

แนวข้อสอบพร้อมเฉลย

ส่วนนี้ออกแบบให้ครอบคลุมทุกบท มีคำตอบ วิธีคิด เหตุผล และระดับความยาก

A. ประเมิน 30 ข้อ

ข้อ 1 (ง่าย)

Memory hierarchy มีเป้าหมายหลักเพื่ออะไร?

- A. ทำให้ CPU ทำงานช้าลง
- B. สร้างสมดุลระหว่าง speed, size, cost
- C. แทนที่ disk ทั้งหมด
- D. ทำให้ทุก memory มีราคาเท่ากัน

เฉลย: B - Hierarchy ใช้หน่วยความจำหลายระดับเพื่อให้ระบบเร็วและจุได้มากในราคาที่เหมาะสม ตัวเลือก A,C,D ผิดเพราะขัดกับเป้าหมาย

ข้อ 2 (ง่าย)

หน่วยความจำใดอยู่ใกล้ CPU และเร็วที่สุด?

- A. Disk
- B. Main Memory
- C. Register
- D. ROM

เฉลย: C - Register อยู่ใน/ใกล้ CPU มากที่สุด Disk/RAM/ROM ช้ากว่า

ข้อ 3 (ง่าย)

k-bit address bus address ได้กี่ locations?

- A. k^2
- B. 2^k
- C. $k \times 8$
- D. $2k$

เฉลย: B - แต่ละบิตมี 2 ค่า รวม k บิตจึงมี 2^k combinations

ข้อ 4 (ง่าย)

MAR ทำหน้าที่อะไร?

- A. เก็บข้อมูลที่จะคำนวณ
- B. เก็บ address ที่จะเข้าถึง memory
- C. เก็บผลลัพธ์ ALU

- D. แปลง virtual address

เฉลย: B - MAR = Memory Address Register ใช้เก็บ address

ข้อ 5 (ง่าย)

MDR ทำหน้าที่อะไร?

- A. เก็บ data ที่อ่าน/เขียน memory
- B. เลือก word line
- C. คำนวณ tag
- D. เก็บ page table

เฉลย: A - MDR = Memory Data Register เก็บข้อมูลที่แลกเปลี่ยนกับ memory

ข้อ 6 (ง่าย)

RAM ต่างจาก serial storage อย่างไร?

- A. เข้าถึงทุกตำแหน่งด้วยเวลาคงที่
- B. อ่านได้อย่างเดียว
- C. ไม่ต้องใช้ไฟ
- D. เก็บข้อมูลถาวรเสมอ

เฉลย: A - RAM random access ไม่ต้องไล่ลำดับเหมือน tape

ข้อ 7 (ง่าย)

ROM เป็น memory แบบใด?

- A. Volatile
- B. Non-volatile
- C. ใช้เป็น cache เสมอ
- D. ต้อง refresh ตลอด

เฉลย: B - ROM เก็บข้อมูลแม้ปิดไฟ

ข้อ 8 (ง่าย)

SRAM เหมาะใช้กับอะไรที่สุด?

- A. Cache
- B. Hard disk
- C. Optical disc
- D. Tape

เฉลย: A - SRAM เร็วแต่แพง จึงใช้ใน cache

ข้อ 9 (ง่าย)

DRAM ต้อง refresh เพราะอะไร?

- A. มี tag ไม่พอ
- B. capacitor รั่วประจุ
- C. ไม่มี address bus
- D. เป็น ROM

เฉลย: B - DRAM เก็บข้อมูลเป็นประจุใน capacitor ที่รั่วได้

ข้อ 10 (ง่าย)

Cache มีจุดประสงค์หลักเพื่ออะไร?

- A. เพิ่ม disk space
- B. ทำให้ main memory ดูเร็วขึ้นต่อ CPU
- C. ลบ page table
- D. แทนที่ register

เฉลย: B - cache เก็บส่วนที่ใช้บ่อยเพื่อช่วยลด average access time

ข้อ 11 (ง่าย)

Temporal locality หมายถึงอะไร?

- A. ข้อมูลใกล้กันจะถูกใช้
- B. สิ่งที่เราเพิ่งใช้มีโอกาสถูกใช้ซ้ำ
- C. ข้อมูลเก็บถาวร
- D. address เพิ่มขึ้นทีละ 1 เสมอ

เฉลย: B - Temporal เกี่ยวกับเวลาและการใช้ซ้ำ

ข้อ 12 (ง่าย)

Spatial locality หมายถึงอะไร?

- A. สิ่งใกล้ตำแหน่งที่เราเพิ่งใช้มีโอกาสถูกใช้
- B. สิ่งที่เราเก็บบน disk เร็วขึ้น
- C. ใช้ register ซ้ำ
- D. write-back เสมอ

เฉลย: A - Spatial เกี่ยวกับตำแหน่งใกล้เคียง

ข้อ 13 (กลาง)

Cache hit คืออะไร?

- A. ข้อมูลอยู่ใน cache
- B. ข้อมูลอยู่ใน disk เท่านั้น
- C. เกิด page fault
- D. cache ถูกปิด

เฉลย: A - hit คือพบข้อมูลใน cache

ข้อ 14 (กลาง)

Write-back ใช้ bit ไດบอกว่า block ถูกแก้แล้ว?

- A. Valid bit
- B. Dirty bit
- C. Offset bit
- D. Sign bit

เฉลย: B - dirty/modified bit ใช้บอกการแก้ไขที่ยังไม่เขียนกลับ

ข้อ 15 (กลาง)

Direct mapping วาง memory block j ที่ใด?

- A. ตำแหน่งใดก็ได้
- B. $j \bmod$ จำนวน cache blocks
- C. $j \times$ จำนวน cache blocks
- D. เฉพาะ disk

เฉลย: B - ตามสูตร direct mapping

ข้อ 16 (กลาง)

Fully associative mapping มีข้อดีหลักคืออะไร?

- A. ยืดหยุ่นที่สุด
- B. ใช้ tag น้อยที่สุดเสมอ
- C. ไม่มี comparator
- D. ไม่ต้องมี valid bit

เฉลย: A - block ลงได้ทุก cache line

ข้อ 17 (กลาง)

k-way set associative หมายถึงอะไร?

- A. มี k caches
- B. หนึ่ง set มี k cache blocks/lines

- C. page size = k
- D. tag มี k bits เสมอ

เฉลย: B - k-way คือจำนวน lines ต่อ set

ข้อ 18 (กลาง)

Valid bit ใช้บอกอะไร?

- A. ข้อมูลใน cache line ใช้ได้หรือไม่
- B. ข้อมูลถูกแก้แล้ว
- C. page size
- D. hit rate

เฉลย: A - valid bit ตรวจสอบว่าข้อมูลใน line ถูกต้อง/มีอยู่จริง

ข้อ 19 (กลาง)

LRU เลือก block ไหนแทนที่?

- A. block ที่ใหญ่ที่สุด
- B. block ที่ไม่ได้ใช้นานที่สุด
- C. block ที่ใหม่ที่สุด
- D. block ที่ tag เป็น 0

เฉลย: B - Least Recently Used = ใช้ล่าสุดนานที่สุด

ข้อ 20 (กลาง)

Miss penalty คืออะไร?

- A. จำนวน hit ทั้งหมด
- B. เวลาที่เสียเมื่อเกิด miss
- C. ขนาด cache
- D. จำนวน tag bits

เฉลย: B - miss penalty คือเวลาชดเชยที่ต้องไปค้นซ้ำกว่า

ข้อ 21 (กลาง)

Virtual memory แก้ปัญหาใดเด่นที่สุด?

- A. CPU ไม่มี ALU
- B. RAM ไม่พอและ address isolation
- C. cache tag ยาว
- D. ROM เขียนยาก

เฉลย: B - virtual memory ช่วย capacity และ isolation

ข้อ 22 (กลาง)

Logical address คืออะไร?

- A. address จริงใน RAM
- B. address ที่โปรแกรมใช้
- C. ตำแหน่ง physical disk sector เท่านั้น
- D. ข้อมูลใน MDR

เฉลย: B - LA/VA คือ address จากมุมโปรแกรม

ข้อ 23 (กลาง)

Physical address คืออะไร?

- A. address จริงใน RAM
- B. address ที่ compiler เห็นเท่านั้น
- C. ชื่อไฟล์
- D. cache tag

เฉลย: A - PA คือตำแหน่งจริงใน main memory

ข้อ 24 (กลาง)

MMU ทำหน้าที่อะไร?

- A. แปลง address และคุม memory access
- B. เพิ่ม clock speed
- C. เขียน ROM
- D. คำนวณ floating point

เฉลย: A - MMU จัดการ virtual memory/address translation

ข้อ 25 (ยาก)

Page คืออะไร?

- A. cache line เสมอ
- B. ชิ้นขนาดคงที่ของ virtual address space
- C. register ชนิดหนึ่ง
- D. bus signal

เฉลย: B - page เป็นหน่วยใน paging

ข้อ 26 (ยาก)

Frame คืออะไร?

- A. ช่องใน physical memory สำหรับเก็บ page
- B. tag ของ cache
- C. byte แรกของ address
- D. ROM cell

เฉลย: A - frame อยู่ใน RAM

ข้อ 27 (ยาก)

Offset คืออะไร?

- A. ตำแหน่งภายใน page/block
- B. หมายเลข disk
- C. hit rate
- D. ชนิดของ bus

เฉลย: A - offset เลือกตำแหน่งย่อยภายใน page/block

ข้อ 28 (ยาก)

Page fault เกิดเมื่อใด?

- A. page ที่ต้องการไม่อยู่ใน RAM หรือ invalid
- B. cache hit
- C. MDR เต็ม
- D. ROM ถูกอ่าน

เฉลย: A - present/valid bit เป็น 0 ทำให้เกิด fault

ข้อ 29 (ยาก)

TLB คืออะไร?

- A. cache สำหรับ address translation
- B. ROM แบบใหม่
- C. main memory module
- D. dirty bit

เฉลย: A - TLB เก็บ VA→PA ล่าสุด

ข้อ 30 (ยาก)

Cache miss ต่างจาก page fault อย่างไร?

- A. cache miss มักดึงจาก RAM แต่ page fault ต้องพึ่ง OS/disk
- B. เหมือนกันทุกอย่าง
- C. page fault เร็วกว่าเสมอ
- D. cache miss เกี่ยวกับ ROM เท่านั้น

เฉลย: A - cache miss อยู่ระหว่าง cache-RAM ส่วน page fault เกี่ยวกับ RAM-disk/OS

B. เต็มคำ 15 ข้อ

1. ถ้า address bus มี k bits จำนวนตำแหน่งที่ address ได้คือ _____.

คำตอบ/วิธีคิด: 2^k | หัวข้อ: บท 2 | ระดับ: ง่าย

2. Register ที่เก็บ address สำหรับ memory access คือ _____.

คำตอบ/วิธีคิด: MAR | หัวข้อ: บท 2 | ระดับ: ง่าย

3. Register ที่เก็บข้อมูลอ่าน/เขียนกับ memory คือ _____.

คำตอบ/วิธีคิด: MDR | หัวข้อ: บท 2 | ระดับ: ง่าย

4. RAM เป็นหน่วยความจำแบบ _____ เพราะข้อมูลหายเมื่อปิดไฟ.

คำตอบ/วิธีคิด: volatile | หัวข้อ: บท 3 | ระดับ: ง่าย

5. ROM เป็นหน่วยความจำแบบ _____.

คำตอบ/วิธีคิด: non-volatile | หัวข้อ: บท 3 | ระดับ: ง่าย

6. SRAM ไม่ต้อง _____ แต่ DRAM ต้อง.

คำตอบ/วิธีคิด: refresh | หัวข้อ: บท 3 | ระดับ: ง่าย

7. หลักที่บอกว่าข้อมูลที่เพิ่งใช้มีโอกาสถูกใช้ซ้ำคือ _____.

คำตอบ/วิธีคิด: Temporal locality | หัวข้อ: บท 4 | ระดับ: ง่าย

8. หลักที่บอกว่าตำแหน่งใกล้เคียงกันมีโอกาสถูกใช้ตามมาคือ _____.

คำตอบ/วิธีคิด: Spatial locality | หัวข้อ: บท 4 | ระดับ: ง่าย

9. Write-back ใช้ _____ bit เพื่อบอกว่าข้อมูลถูกแก้แล้ว.

คำตอบ/วิธีคิด: dirty/modified | หัวข้อ: บท 4 | ระดับ: กลาง

10. Direct mapping ใช้สูตร cache block = memory block _____ number of cache blocks.

คำตอบ/วิธีคิด: mod | หัวข้อ: บท 5 | ระดับ: กลาง

11. ใน set-associative cache หน่วยกลุ่มของ cache lines เรียกว่า _____.

คำตอบ/วิธีคิด: set | หัวข้อ: บท 5 | ระดับ: กลาง

12. Miss rate เท่ากับ _____ เมื่อ hit rate = h .

คำตอบ/วิธีคิด: $1 - h$ | หัวข้อ: บท 6 | ระดับ: กลาง

13. Address ที่โปรแกรมใช้เรียกว่า Logical Address หรือ _____.

คำตอบ/วิธีคิด: Virtual Address | หัวข้อ: บท 7 | ระดับ: กลาง

14. ตารางที่ map page number ไป frame number คือ _____.

คำตอบ/วิธีคิด: Page table | หัวข้อ: บท 8 | ระดับ: กลาง

15. Cache สำหรับผลการแปลง address เรียกว่า _____.

คำตอบ/วิธีคิด: TLB | หัวข้อ: บท 8 | ระดับ: กลาง

C. อธิบายสั้น 15 ข้อ

1. ทำไม CPU ที่เร็วมากยังอาจรอ memory?

แนวคำตอบ: เพราะคำสั่งและข้อมูลต้องถูก fetch/load จาก memory ถ้า memory ช้า CPU จะ stall หรือรอข้อมูล | ระดับ: ง่าย

2. ทำไมต้องมี cache?

แนวคำตอบ: เพื่อลดเวลาเฉลี่ยในการเข้าถึง memory โดยเก็บข้อมูล/คำสั่งที่ใช้บ่อยไว้ใกล้ CPU | ระดับ: ง่าย

3. อธิบาย temporal locality พร้อมตัวอย่าง

แนวคำตอบ: ข้อมูลหรือ instruction ที่เพิ่งใช้มีโอกาสถูกใช้ซ้ำ เช่น loop ใช้ instruction เดิมหลายรอบ | ระดับ: ง่าย

4. อธิบาย spatial locality พร้อมตัวอย่าง

แนวคำตอบ: ตำแหน่งใกล้กับที่เพิ่งใช้มีโอกาสถูกใช้ตามมา เช่น อ่าน array ตามลำดับ | ระดับ: ง่าย

5. Write-through ต่างจาก write-back อย่างไร?

แนวคำตอบ: write-through เขียน cache และ memory พร้อมกัน ส่วน write-back เขียน cache ก่อนและเขียนกลับ memory เมื่อ block ถูกแทนที่ | ระดับ: ง่าย

6. Valid bit ต่างจาก Dirty bit อย่างไร?

แนวคำตอบ: valid bit บอกว่าข้อมูลใน line ใช้ได้หรือไม่ dirty bit บอกว่าถูกแก้ไขแล้วแต่ยังไม่เขียนกลับ | ระดับ: กลาง

7. ทำไม direct mapping อาจเกิด collision?

แนวคำตอบ: เพราะหลาย memory blocks ถูกบังคับให้ลง cache line เดียวจาก modulo mapping | ระดับ: กลาง

8. LRU ใช้แนวคิด locality อย่างไร?

แนวคำตอบ: เชื่อว่า block ที่ไม่ได้ใช้นานที่สุดมีโอกาสใช้น้อยกว่าบล็อกที่เพิ่งใช้ | ระดับ: กลาง

9. Virtual memory ให้ประโยชน์อะไรกับโปรแกรมเมอร์?

แนวคำตอบ: ทำให้เห็น address space ใหญ่และเป็นของตัวเอง ไม่ต้องรู้ layout จริงใน RAM | ระดับ: กลาง

10. MMU สำคัญอย่างไร?

แนวคำตอบ: แปลง virtual/logical address เป็น physical address และช่วยตรวจสอบสิทธิ์การเข้าถึง | ระดับ: กลาง

11. Page fault คืออะไร?

แนวคำตอบ: เหตุการณ์เมื่อ page ที่ต้องการไม่อยู่ใน RAM ต้องให้ OS โหลดจาก disk | ระดับ: กลาง

12. Demand paging คืออะไร?

แนวคำตอบ: โหลด page เข้า RAM เฉพาะเมื่อถูกใช้งานจริง | ระดับ: กลาง

13. TLB ช่วยอะไร?

แนวคำตอบ: ลดเวลาการแปลง address โดย cache ผล VA→PA ที่ใช้ล่าสุด | ระดับ: ยาก

14. Cache miss กับ page fault ต่างกันอย่างไร?

แนวคำตอบ: cache miss ดึง block จาก RAM; page fault ต้องให้ OS ดึง page จาก disk และแพงกว่ามาก | ระดับ: ยาก

15. ทำไม page offset ไม่เปลี่ยนตอนแปลง VA เป็น PA?

แนวคำตอบ: เพราะ page และ frame มีขนาดเท่ากัน จึงเปลี่ยนเฉพาะ page number เป็น frame number | ระดับ: ยาก

D. คำนวณ 10 ข้อ

1. Address bus 20 bits และแต่ละ location = 1 byte มี memory กี่ MB?

วิธีคิด/คำตอบ: 2^{20} bytes = 1,048,576 bytes = 1 MB | ระดับ: ง่าย

2. RAM 128×8 มีความจุกี่ bits และกี่ bytes?

วิธีคิด/คำตอบ: $128 \times 8 = 1024$ bits = 128 bytes | ระดับ: ง่าย

3. RAM 1024×1 มีความจุกี่ bits?

วิธีคิด/คำตอบ: $1024 \times 1 = 1024$ bits | ระดับ: ง่าย

4. Cache block มี 16 words ต้องใช้ word/offset กี่ bits?

วิธีคิด/คำตอบ: $\log_2(16)=4$ bits | ระดับ: ง่าย

5. Main memory block 130 map ไป cache block ไต ถ้า direct cache มี 128 blocks?

วิธีคิด/คำตอบ: $130 \bmod 128 = 2$ | ระดับ: กลาง

6. Memory block 130 map ไป set ไต ถ้า cache มี 64 sets?

วิธีคิด/คำตอบ: $130 \bmod 64 = 2$ | ระดับ: กลาง

7. 16-bit address, block size 16 words, direct cache 128 blocks แบ่ง field อย่างไร?

วิธีคิด/คำตอบ: word=4 bits, block=7 bits, tag=16-4-7=5 bits | ระดับ: กลาง

8. $h=0.9$, $t_C=1$, $t_M=10$ หา t_{avg}

วิธีคิด/คำตอบ: $0.9 \times 1 + 0.1 \times 10 = 1.9$ cycles | ระดับ: กลาง

9. $h=0.98$, $t_C=2\text{ns}$, $t_M=80\text{ns}$ หา t_{avg}

วิธีคิด/คำตอบ: $0.98 \times 2 + 0.02 \times 80 = 3.56$ ns | ระดับ: ยาก

10. VA 0x1234, page size 1KB, page 4 → frame 9 หา PA

วิธีคิด/คำตอบ: VA=4660, offset=4660 mod 1024=564, PA=9×1024+564=9780=0x2634 | ระดับ: ยาก

E. วิเคราะห์สถานการณ์ 5 ข้อ

1. โปรแกรมวน loop เล็ก ๆ ซ้ำหลายล้านครั้ง cache จะช่วยได้อย่างไร?

วิเคราะห์/คำตอบ: Temporal locality สูง instruction และ data ใน loop จะอยู่ใน cache ทำให้ hit rate สูงและลด access time | ระดับ: กลาง

2. ระบบมี cache hit rate สูงมาก แต่ยังช้า อาจเกิดจากอะไร?

วิเคราะห์/คำตอบ: hit time สูง, miss penalty สูงมาก, cache ใหญ่เกินไปจนค้นหาช้า หรือ workload มี page fault มาก | ระดับ: กลาง

3. โปรแกรมสองตัวใช้ logical address 0x1024 เหมือนกัน ทำไมไม่ชนกัน?

วิเคราะห์/คำตอบ: แต่ละโปรแกรมมี virtual address space และ page table ของตัวเอง MMU map ไปคนละ physical frames | ระดับ: กลาง

4. ทำไม page fault ส่งผลหนักกว่า cache miss?

วิเคราะห์/คำตอบ: เพราะ page fault ต้องเข้าสู่ OS และอาจอ่าน disk ซึ่งช้ากว่า RAM มาก | ระดับ: ยาก

5. ถ้า direct cache มี collision บ่อย ควรปรับอะไร?

วิเคราะห์/คำตอบ: เพิ่ม associativity เช่น 2-way/4-way set associative หรือปรับขนาด cache/block ให้เหมาะกับ locality | ระดับ: ยาก

Final Cheat Sheet 1-2 หน้า

เรื่อง	จำให้ได้
Hierarchy	Register เร็ว/เล็ก/แพง → Disk ช้า/ใหญ่/ถูก
Address bits	$k \text{ bits} = 2^k \text{ locations}$
Read memory	MAR ใส่ address → Read → memory ส่ง data → MFC → MDR
Write memory	MAR ใส่ address + MDR ใส่ data → Write
SRAM vs DRAM	SRAM เร็วไม่ refresh ใช้ cache; DRAM หนาแน่นถูกต้อง refresh ใช้ main memory
Cache locality	Temporal = ใช้ซ้ำเร็ว ๆ นี้, Spatial = ใช้ตำแหน่งใกล้กัน
Hit/Miss	Hit อ่านเร็วใน cache; Miss ต้องดึง block จาก RAM
Write policies	Write-through ง่ายแต่เขียนบ่อย; Write-back ใช้ dirty bit
Direct mapping	$\text{line} = \text{block} \bmod \text{cache_lines}$
Set associative	$\text{set} = \text{block} \bmod \text{number_of_sets}$, $k\text{-way} = k \text{ lines/set}$
AMAT/ t_{avg}	$t_{avg} = h \cdot t_C + (1-h) \cdot t_M$
Virtual memory	โปรแกรมเห็น VA/LA; MMU แปลงเป็น PA
Paging	$VA = \text{page number} + \text{offset}$; $PA = \text{frame number} + \text{offset}$
Page fault	page ไม่อยู่ RAM → OS โหลดจาก disk
TLB	cache ของ address translation

ก่อนเข้าห้องสอบ: ถ้าข้อสอบมีตัวเลข ให้เริ่มจากดูหน่วยก่อนเสมอ: bits/bytes, words/blocks/pages/frames, hit rate เป็นทศนิยมหรือเปอร์เซ็นต์ และ tM ในโจทย์หมายถึงอะไร

แหล่งข้อมูลจากไฟล์ Lecture

- Lecture 9_MemorySystem: Lecture9_MemorySystem.pdf
- Lecture 10_CacheMemory: Lecture10_CacheMemory.pdf
- Lecture 11_VirtualMemory: Lecture11_VirtualMemory.pdf
- Lecture 11_extended: Lecture11_extended.pdf

หมายเหตุ: เอกสารนี้เรียบเรียงใหม่จากสไลด์และเติมคำอธิบายพื้นฐานเพื่อให้อ่านสอบได้ต่อเนื่อง ไม่ได้คัดลอกข้อความจากสไลด์แบบตรง ๆ ทั้งหมด